



COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S1
DATE PERFORMED:	July 28, 2026	DATE SUBMITTED:	August 1, 2026
NAME:	Mendoza, Ryo Shin P.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL:

SOURCE CODE

```
#include <iostream>
#include <stack> // call using stack from STL

int main() {
    // create an object for the stack
    std::stack<int> newStack;

    std::cout<<"Stack empty? " << newStack.empty()<<std::endl;
    //push to the stack
    /*15
    6
    3*/
    newStack.push(3);
    newStack.push(6);
    newStack.push(15);

    std::cout<<"Stack empty? " << newStack.empty()<<std::endl;
    std::cout << "The size of the stack: " << newStack.size() <<
std::endl;
    std::cout << "The top of the element is: " << newStack.top() <<
std::endl;
    std::cout << "Removing elements from the stack: \n";
    newStack.pop();
```

```
std::cout << "The top of the element is: " << newStack.top() <<
std::endl;
newStack.pop();

std::cout << "The top of the element is: " << newStack.top() <<
std::endl;
newStack.pop();

std::cout<<"Stack empty? " << newStack.empty()<<std::endl;
std::cout << "The size of the stack: " << newStack.size() <<
std::endl;

return 0;
}
```

Table 4-1-1. Source Code of ILO A

OUTPUT

```
Stack empty? 1
Stack empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 6
The top of the element is: 3
Stack empty? 1
The size of the stack: 0
```

Table 4-1-2. Output of ILO A

CODE ANALYSIS

This program includes the library STL of stack, such as pop, top, empty, and size. The pop() removes the value of stack, the top checks what is the value of the top, while empty and size checks the values of stack if it has a value or not.

Table 4-1-3. Code Analysis of ILO A

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack:

SOURCE CODE

```
#include <iostream>

using namespace std;

// Maximum capacity of the stack
const size_t maxCap = 100;

// Stack array
int stack[maxCap];
int top = -1;

// Variables
int i; // User-defined stack size
int choice;
int newData;

// Function prototypes
void push();
void Top();
void pop();
bool isEmpty();
bool isFull();
void display();

int main()
{
    cout << "Enter the number of max elements for the new Stack: ";
    cin >> i;

    // Prevent invalid size
    if (i > maxCap || i <= 0)
    {
        cout << "Invalid stack size!" << endl;
```

```
    return 0;
}

while (true)
{
    cout << "\n==== Stack Operation =====" << endl;
    cout << "1. PUSH" << endl;
    cout << "2. POP" << endl;
    cout << "3. TOP" << endl;
    cout << "4. isEmpty" << endl;
    cout << "5. Display" << endl;
    cout << "6. Exit" << endl;
    cout << "Enter choice: ";
    cin >> choice;

    switch (choice)
    {
        case 1:
            push();
            break;

        case 2:
            pop();
            break;

        case 3:
            Top();
            break;

        case 4:
            if (isEmpty())
                cout << "Stack is empty." << endl;
            else
                cout << "Stack is not empty." << endl;
            break;

        case 5:
```

```
        display();
        break;

    case 6:
        main();

    default:
        cout << "Invalid choice!" << endl;
    }
}

return 0;
}

// Check if stack is empty
bool isEmpty()
{
    return (top == -1);
}

// Check if stack is full
bool isFull()
{
    return (top == i - 1);
}

// Push operation
void push()
{
    if (isFull())
    {
        cout << "Stack Overflow!" << endl;
        return;
    }

    cout << "New value to add into the stack: ";
    cin >> newData;
```

```
    stack[++top] = newData;
}

// Display top element
void Top()
{
    if (isEmpty())
    {
        cout << "Stack Underflow!" << endl;
        return;
    }

    cout << "The element on the top of the stack is: " << stack[top]
<< endl;
}

// Pop operation
void pop()
{
    if (isEmpty())
    {
        cout << "Stack Underflow!" << endl;
        return;
    }

    cout << "Popping: " << stack[top] << endl;
    top--;
}

// Display function
void display()
{
    if (isEmpty())
    {
        std::cout << "Stack is empty." << endl;
        return;
    }
}
```

```
std::cout << "Stack contents: " << std::endl;

for (int j = top; j >= 0; j--)
{
    std::cout << "|" << stack[j] << "|" << "\n";
}
}
```

Table 4-2-1: Source Code of ILO B

OUTPUT

```
Enter the number of max elements for the new Stack: 2

===== Stack Operation =====
1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit
Enter choice: 1
New value to add into the stack: 7

===== Stack Operation =====
1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit
Enter choice: 1
New value to add into the stack: 6
```

Figure 1. Entering number of stacks and adding the value

```
===== Stack Operation =====
```

```
1. PUSH
```

```
2. POP
```

```
3. TOP
```

```
4. isEmpty
```

```
5. Display
```

```
6. Exit
```

```
Enter choice: 5
```

```
Stack contents:
```

```
|6|
```

```
|7|
```

Figure 2. Display output

```
===== Stack Operation =====
```

```
1. PUSH
```

```
2. POP
```

```
3. TOP
```

```
4. isEmpty
```

```
5. Display
```

```
6. Exit
```

```
Enter choice: 2
```

```
Popping: 6
```

Figure 3. Popping the value from top


```
===== Stack Operation =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit

```
Enter choice: 5  
Stack contents:  
|7|
```

Figure 4. Checking if the stack is popped

```
===== Stack Operation =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit

```
Enter choice: 3  
The element on the top of the stack is: 7
```

Figure 5. Checks the value from top

```
===== Stack Operation =====
1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit
Enter choice: 4
Stack is not empty.
```

Figure 6. Checking the stack if it is empty

```
===== Stack Operation =====
1. PUSH
2. POP
3. TOP
4. isEmpty
5. Display
6. Exit
Enter choice: 6
Enter the number of max elements for the new Stack: █
```

Figure 7. Exiting the program

CODE ANALYSIS

The program uses a stack implemented with a static array in C++. It follows the Last In, First Out (LIFO) rule, which means the last value added is the first one removed. The user enters the stack size and chooses different operations from the menu. The push() function adds a value, pop() removes the top value, Top() shows the top value, and isEmpty() checks if the stack has no elements.

A display() function was added to the program. This function prints all the values currently stored in the stack from top to bottom. It first checks if the stack is empty, then displays each element. This modification makes it easier to see all the data in the stack and check if the push and pop operations are working.

ILOB.2. Stacks using Linked Lists:

Code	Output/Description
<pre>#include <iostream> // class node that contains data and the next pointer template <typename T> class Node { public: T data; Node<T>* next; }; // global pointer for the top of the stack template <typename T> Node<T>* head = nullptr;</pre>	This code shows the creation of the class node that contains data and the next pointer. This will also create a global pointer for the top of the stack.
<pre>//===== // push operation //===== template <typename T> void push(T newData) { // create a new node Node<T>* newNode = new Node<T> // store data newNode->data = newData; // point to current top newNode->next = head<T>; // update top head<T> = newNode; }</pre>	Push Operation (push()) The push operation adds a new element to the top of the stack. It creates a new node, stores the given data, and connects it to the current top of the stack. This will output the character that pushed in the main function. Output: <pre>std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK --- \n"; // push push('C'); push('P'); push('E');</pre> --- PUSHING 'C' 'P' 'E' ONTO THE STACK --- Top of stack: E

```
//=====
// pop operation
//=====
template <typename T>
T pop()
{
    // check if stack is empty
    if (head<T> == nullptr)
    {
        std::cout << "Stack underflow" << std::endl;
        return T();
    }

    // store current top node
    Node<T>* temp = head<T>;

    // save the data
    T tempVal = temp->data;

    // move top to next node
    head<T> = head<T>->next;

    // delete removed node
    delete temp;

    // return removed value
    return tempVal;
}
```

Pop Operation (pop())

The pop operation removes the element at the top of the stack. It gets the data from the top node, moves the top pointer to the next node, and deletes the removed node.

Output:

```
std::cout << "\n -----Popping all the elements ----- \n";

while (!isEmpty<char>())
{
    std::cout << "Popped: " << pop<char>() << std::endl;
}

std::cout << "\n -----Testing the Underflow ----- \n";
pop<char>();
```

```
-----Popping all the elements -----
Popped: E
Popped: P
Popped: C
```

```
//=====
// display top operation
//=====
template <typename T>
void Top()
{
    if (head<T> == nullptr)
    {
        std::cout << "Stack underflow" << std::endl;
        return;
    }

    std::cout << "Top of stack: " << head<T>->data << std::endl;
}
```

Top Operation (Top())

The top operation displays the element currently placed at the top of the stack without removing it. It helps identify which element will be removed next.

Output:

```
// display the current top
Top<char>(); // top should be 'E'
```

```
//=====
// check if empty
//=====
template <typename T>
bool isEmpty()
{
    return head<T> == nullptr;
}
```

IsEmpty Operation (isEmpty())

The isEmpty operation checks if the stack contains any elements. It returns true when the stack is empty and false when there are elements inside the stack.

```
-----Display all-----
Stack is empty
```

```
//=====
// display all stack elements
//=====
template <typename T>
void display()
{
    if (head<T> == nullptr)
    {
        std::cout << "Stack is empty" << std::endl;
        return;
    }

    Node<T>* current = head<T>;

    std::cout << "Stack elements: ";

    while (current != nullptr)
    {
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << "\n";
}
```

Display Operation (display())

The display operation shows all the elements stored in the stack. It starts from the top element and continues displaying each node until the end of the stack.

```
std::cout << "\n-----Display all-----\n";
display<char>();
```

Code Analysis/Reflection

This activity helped me understand how a stack works using a linked list in C++. I learned how to use the push, pop, Top, display, and isEmpty functions. At first, I encountered some coding errors, but fixing them helped me understand pointers and stack operations better. Adding the display() function also made it easier to see the elements in the stack. Overall, this activity improved my C++ programming skills and helped me understand the LIFO (Last In, First Out) concept.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

SOURCE CODE

Main.cpp

```
#include <iostream>
#include <string>
#include "Stackarray.h"
```

```
// Check if symbols match
bool isPair(char open, char close)
{
    return (open == '(' && close == ')') || (open == '[' && close ==
']') || (open == '{' && close == '}');
}

// Check if expression is balanced
bool checkBalance(std::string expression)
{
    // Reset stack
    top = -1;

    for (int i = 0; i < expression.length(); i++)
    {
        char ch = expression[i];

        // Opening symbols
        if (ch == '(' || ch == '[' || ch == '{')
        {
            push(ch);
        }

        // Closing symbols
        else if (ch == ')' || ch == ']' || ch == '}')
        {
            if (isEmpty())
            {
                return false;
            }

            char temp = pop();

            if (!isPair(temp, ch))
            {

```

```
        return false;
    }
}

return isEmpty();
}

int main()
{
    std::string expression;

    std::cout << "=====" << std::endl;
    std::cout << "        Symbol Balance Checker" << std::endl;
    std::cout << "=====" << std::endl;

    std::cout << "Enter an expression: ";
    std::getline(std::cin, expression);

    if (checkBalance(expression))
    {
        std::cout << "\nExpression is VALID." << std::endl;
    }
    else
    {
        std::cout << "\nExpression is INVALID." << std::endl;
    }

    return 0;
}
```

StackArray.h

```
#ifndef STACK_ARRAY_H
#define STACK_ARRAY_H

#include <iostream>

const int MAX = 100;

// Stack array
char stack[MAX];
int top = -1;

//=====
// Push
//=====
void push(char data)
{
    if (top == MAX - 1)
    {
        std::cout << "Stack Overflow" << std::endl;
        return;
    }

    top++;
    stack[top] = data;
}

//=====
// Pop
//=====
char pop()
{
    if (top == -1)
    {
        std::cout << "Stack Underflow" << std::endl;
        return '\0';
    }
}
```



```

}

char value = stack[top];
top--;

return value;
}

//=====
// Is Empty
//=====
bool isEmpty()
{
    return top == -1;
}

#endif

```

Expression	Valid? (Y / N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	<pre> ===== Symbol Balance Checker ===== Enter an expression: (A+B)+(C-D) Expression is VALID. </pre>	The program identified that all opening and closing parentheses matched correctly. Since no unmatched symbols remained in the stack, the expression is valid.
((A+B)+(C-D)	N	<pre> ===== Symbol Balance Checker ===== Enter an expression: ((A+B)+(C-D) Expression is INVALID. </pre>	The program found one extra opening parenthesis (that did not have a matching closing parenthesis). Because the stack was not empty at the end, the expression is invalid.

$((A+B)+[C-D])$	Y	<pre>===== Symbol Balance Checker ===== Enter an expression: ((A+B)+[C-D]) Expression is VALID.</pre>	The program correctly matched all pairs of parentheses () and brackets []. Since every opening symbol had the correct closing symbol, the expression is valid.
$((A+B)+[C-D])\}$	N	<pre>===== Symbol Balance Checker ===== Enter an expression: ((A+B)+[C-D])\} Expression is INVALID.</pre>	The program detected a mismatched closing symbol. The opening bracket [was closed with } instead of], making the expression invalid.

Code Analysis/Reflection

In this activity, I used stack using arrays and it helped me understand how a stack can be used to check if symbols in an expression are balanced. I learned how the push and pop operations work together to compare opening and closing symbols. While making the program, I fixed some coding errors and gained a better understanding of stack operations and arrays. Overall, this activity improved my C++ programming skills and showed me how stacks can be used to solve real programming problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance, grammar correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.